Class Activity Exercise: Data Warehouse Techniques

Objective: To understand ETL, data modeling, and querying for decision-making.

Activity Title: Designing and Exploring a Data Warehouse

```
# Install Kaggle API
!pip install kaggle
# Standard libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
Defaulting to user installation because normal site-packages is not wri
Collecting kaggle
  Downloading kaggle-2.0.0-py3-none-any.whl.metadata (15 kB)
Requirement already satisfied: bleach in ./.local/lib/python3.14/site-p
Collecting kagglesdk<1.0,>=0.1.15 (from kaggle)
  Downloading kagglesdk-0.1.16-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: packaging in ./.local/lib/python3.14/sit
Requirement already satisfied: protobuf in /usr/lib/python3.14/site-pac
Requirement already satisfied: python-dateutil in ./.local/lib/python3.
Collecting python-slugify (from kaggle)
  Downloading python_slugify-8.0.4-py2.py3-none-any.whl.metadata (8.5 k
Requirement already satisfied: requests in ./.local/lib/python3.14/site
Collecting tqdm (from kaggle)
  Downloading tqdm-4.67.3-py3-none-any.whl.metadata (57 kB)
Requirement already satisfied: urllib3>=1.15.1 in ./.local/lib/python3.
Requirement already satisfied: webencodings in ./.local/lib/python3.14/
Requirement already satisfied: six>=1.5 in ./.local/lib/python3.14/site
Collecting text-unidecode>=1.3 (from python-slugify->kaggle)
  Downloading text_unidecode-1.3-py2.py3-none-any.whl.metadata (2.4 kB)
Requirement already satisfied: charset_normalizer<4,>=2 in ./.local/lib
Requirement already satisfied: idna<4,>=2.5 in ./.local/lib/python3.14/
Requirement already satisfied: certifi>=2017.4.17 in ./.local/lib/pytho
Downloading kaggle-2.0.0-py3-none-any.whl (75 kB)
Downloading kagglesdk-0.1.16-py3-none-any.whl (160 kB)
Downloading python_slugify-8.0.4-py2.py3-none-any.whl (10 kB)
Downloading text_unidecode-1.3-py2.py3-none-any.whl (78 kB)
Downloading tqdm-4.67.3-py3-none-any.whl (78 kB)
Installing collected packages: text-unidecode, tqdm, python-slugify, ka
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5/5 [kaggle]━━━ 4/5 [kaggl
Successfully installed kaggle-2.0.0 kagglesdk-0.1.16 python-slugify-8.0
```

Explanation:

We preview the dataset to understand its structure.Columns show three sources:

```
Sales System → Sales, Profit, Quantity, Discount

Customer Database-Segment, Region, Country, City, State, Postal Code

Inventory/Logistics System-Category, Sub-Category, Ship Mode
```

```
df = pd.read_csv("/home/kelvin/Assignments/Class Activity And Practica
df.head()
```

	Ship Mode	Segment	Country	City	State	Postal Code	Region	Categ
0	Second Class	Consumer	United States	Henderson	Kentucky	42420	South	Furni
1	Second Class	Consumer	United States	Henderson	Kentucky	42420	South	Furni
2	Second Class	Corporate	United States	Los Angeles	California	90036	West	O Supp
3	Standard Class	Consumer	United States	Fort Lauderdale	Florida	33311	South	Furni
4	Standard Class	Consumer	United States	Fort Lauderdale	Florida	33311	South	O Supp

```
print(df.columns)
```

```
Index(['Ship Mode', 'Segment', 'Country', 'City', 'State', 'Postal Code
       'Region', 'Category', 'Sub-Category', 'Sales', 'Quantity', 'Disc
       'Profit'],
      dtype='object')
```

Part 2: ETL Process Design

Explanation:

```
Extract: Data loaded from CSV.

Transform: Missing values replaced, duplicates removed, names cleaned.
```

```
Load: Data kept in memory (in practice, batch load into warehouse).
```

```
# Fill missing values with 0 instead of dropping
df.fillna(0, inplace=True)

# Remove duplicates
df.drop_duplicates(inplace=True)

# Standardize column names (remove spaces)
df.columns = df.columns.str.strip()

# Quick check
print(df.isnull().sum())
```

```
Ship Mode       0
Segment         0
Country         0
City            0
State           0
Postal Code     0
Region          0
Category        0
Sub-Category    0
Sales           0
Quantity        0
Discount        0
Profit          0
dtype: int64
```

Part 3: Data Modeling

Explanation:

```
Fact Table: Sales Fact (Sales, Profit, Quantity, Discount).

Dimensions: Customer, Product, Shipping.

This forms a star schema: fact table in the center, dimensions around it.
```

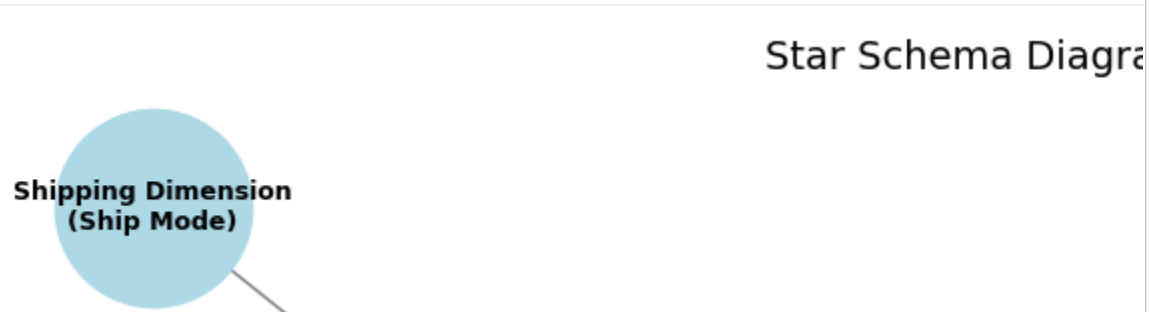
```
# Fact table: sales metrics
fact_sales = df[['Sales','Profit','Quantity','Discount','Region','Cate

# Customer dimension
customer_dim = df[['Segment','Region','Country','City','State','Postal

# Product dimension
```

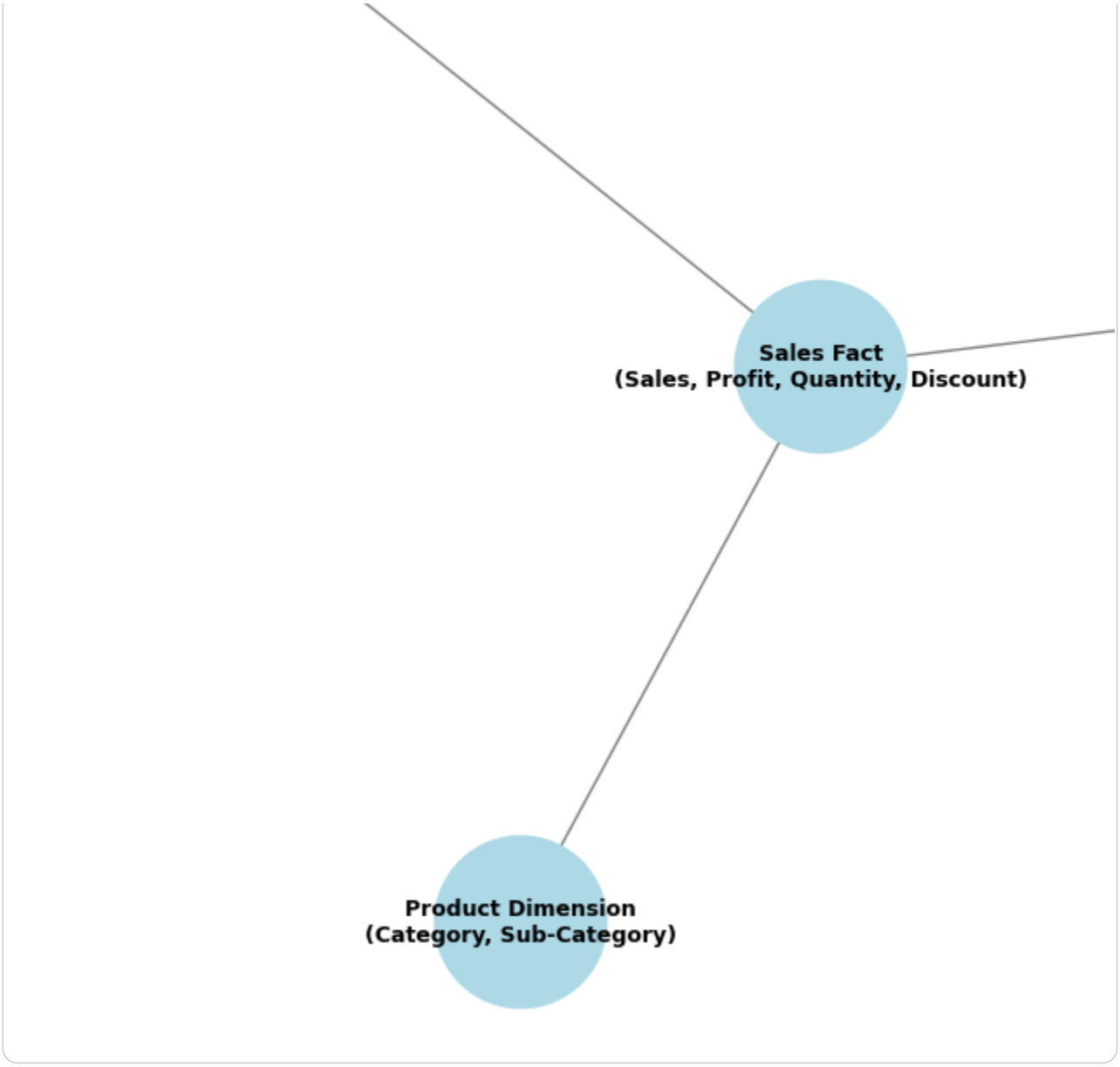
```
product_dim = df[['Category','Sub-Category']].drop_duplicates()

# Shipping dimension
shipping_dim = df[['Ship Mode']].drop_duplicates()
```

```
!pip install graphviz
from graphviz import Digraph
```

```
Defaulting to user installation because normal site-packages is not wri
Requirement already satisfied: graphviz in ./.local/lib/python3.14/site
```

```
import matplotlib.pyplot as plt
import networkx as nx

# Create a graph
G = nx.Graph()

# Add nodes
G.add_node("Sales Fact\n(Sales, Profit, Quantity, Discount)")
G.add_node("Customer Dimension\n(Segment, Region, Country, City, State
G.add_node("Product Dimension\n(Category, Sub-Category)")
G.add_node("Shipping Dimension\n(Ship Mode)")

# Connect dimensions to fact table
G.add_edge("Sales Fact\n(Sales, Profit, Quantity, Discount)",
           "Customer Dimension\n(Segment, Region, Country, City, State
G.add_edge("Sales Fact\n(Sales, Profit, Quantity, Discount)",
           "Product Dimension\n(Category, Sub-Category)")
G.add_edge("Sales Fact\n(Sales, Profit, Quantity, Discount)",
           "Shipping Dimension\n(Ship Mode)")

# Draw the graph
plt.figure(figsize=(10,7))
pos = nx.spring_layout(G, seed=42)  # layout for positioning
nx.draw(G, pos, with_labels=True, node_size=5000, node_color="lightblu
        font_size=9, font_weight="bold", edge_color="gray")
plt.title("Star Schema Diagram", fontsize=14)
plt.show()
```





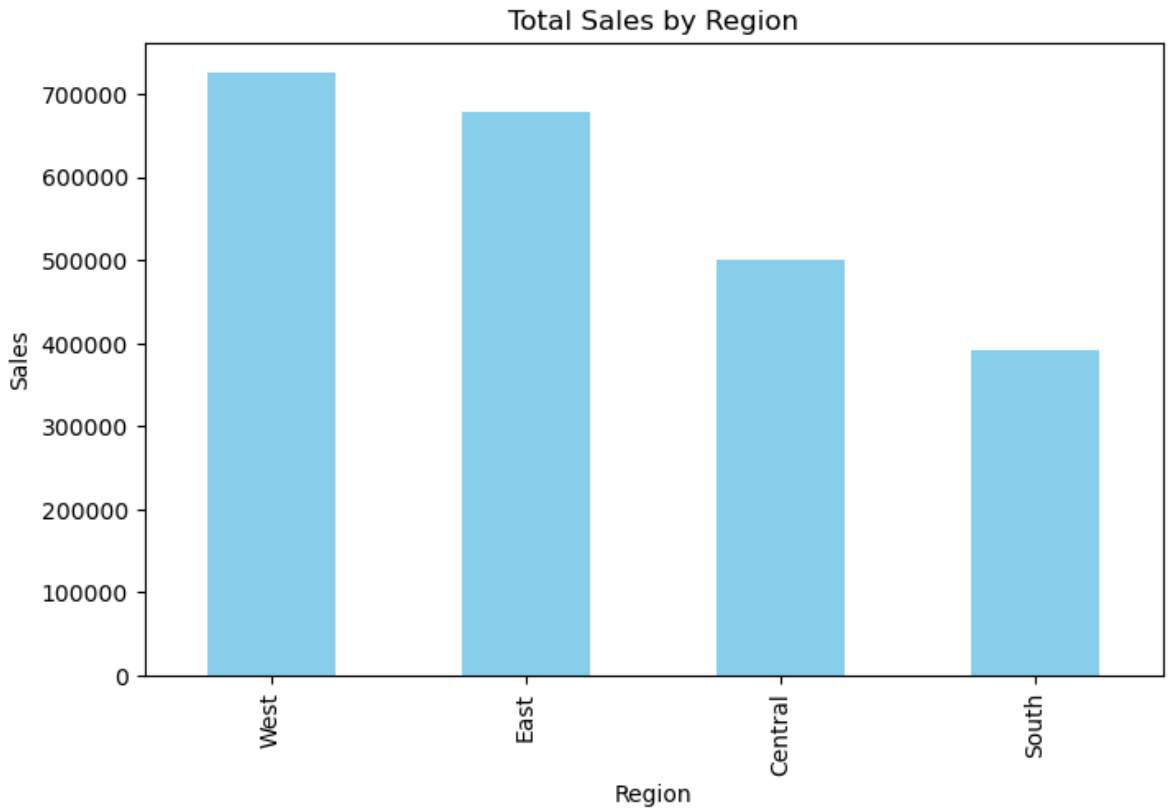
Part 4: Query & Analysis

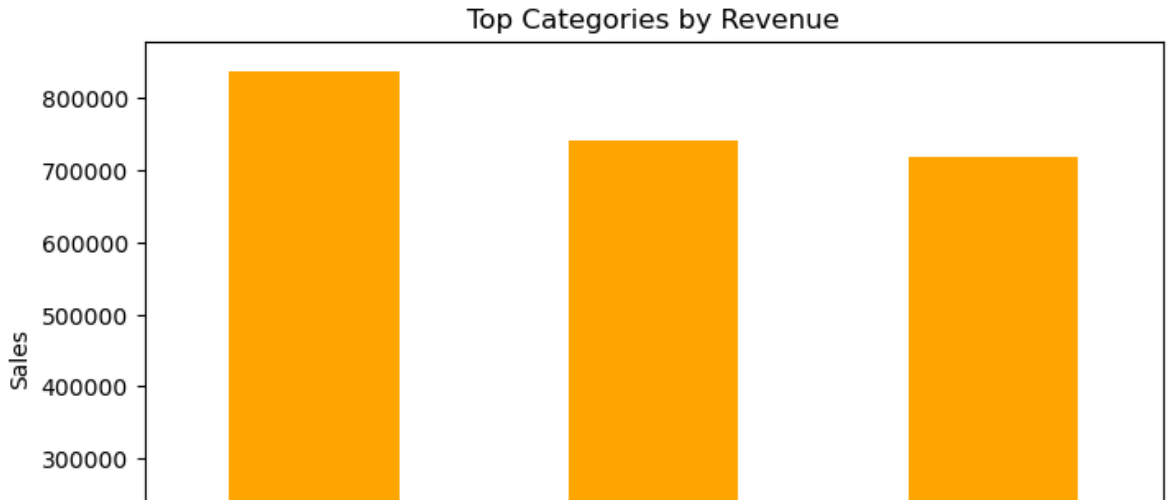
Explanation:

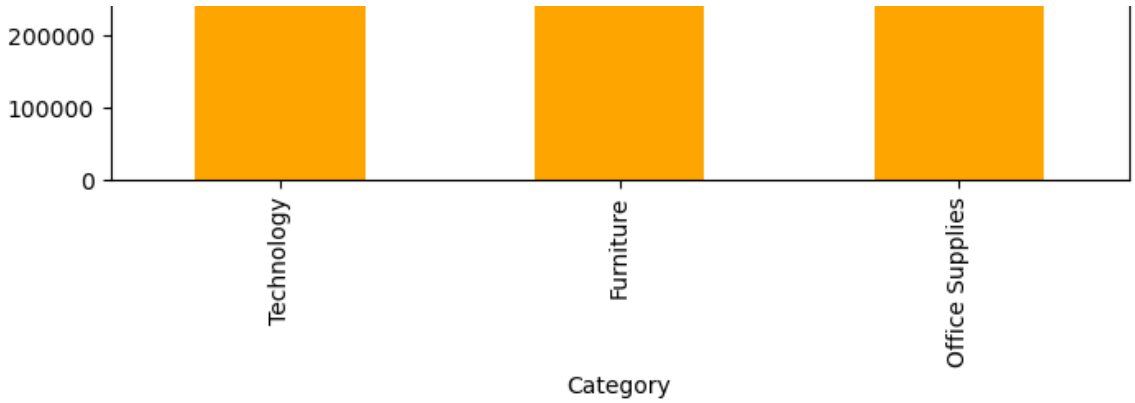
```
Bar charts show regional and category sales.

Pie chart shows sales distribution by customer segment.
```

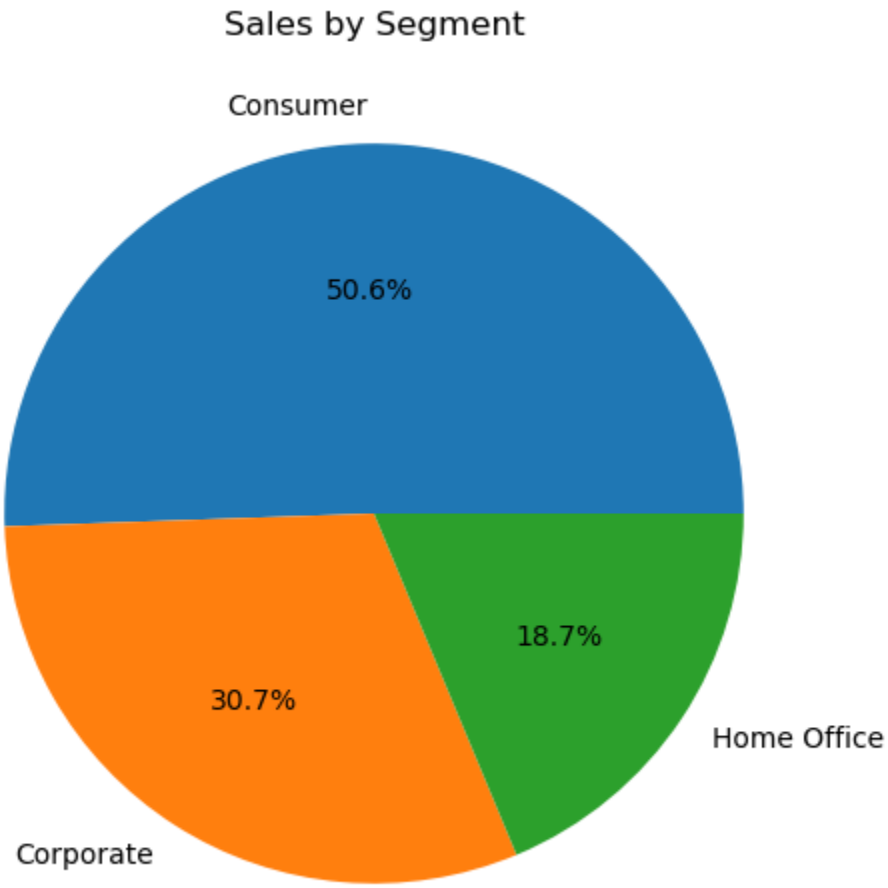
```
import matplotlib.pyplot as plt

# Total sales by region
region_sales = df.groupby('Region')['Sales'].sum().sort_values(ascendi
region_sales.plot(kind='bar', color='skyblue', figsize=(8,5), title="T
plt.ylabel("Sales")
```

```
plt.ylabel("Sales")
plt.show()

# Top categories by revenue
category_sales = df.groupby('Category')['Sales'].sum().sort_values(asc
category_sales.plot(kind='bar', color='orange', figsize=(8,5), title='
plt.ylabel("Sales")
plt.show()

# Segment analysis
segment_sales = df.groupby('Segment')['Sales'].sum()
segment_sales.plot(kind='pie', autopct='%1.1f%%', figsize=(6,6), title
plt.ylabel("")
plt.show()
```









Part 5: Discussion and Reflection

Part 5: Discussion and Reflection

Challenges faced during ETL design:

```
1.Handling missing values without losing important records.

2.Removing duplicates while ensuring no valid transactions were deleted.

3.Standardizing categorical data (e.g., ensuring region names are consist

4.Lack of a time dimension in the dataset, which limited trend analysis o
```

Importance of clean and structured data:

```
1.Clean data prevents misleading results and ensures accuracy in analysis

2.Structured data allows queries to run efficiently and consistently.

3.Properly transformed data supports integration across multiple sources

4.Reliable data builds trust in the warehouse outputs for decision-makers
```

How data warehousing supports decision-making:

```
1.Provides a single source of truth by consolidating data from different

2.Enables managers to identify top-performing regions, categories, and cu

3.Supports trend analysis and forecasting when time dimensions are availa

4.Improves operational efficiency by highlighting areas of high profit or

5.Facilitates strategic planning by giving clear insights into customer b
```

Part 6: Practical Tasks

Explanation:

```
1.confirmed the Segment column exists.

2.ETL fills missing values.
```

```
3.Found the top region by sales.

4.Displayed the star schema diagram.

5.Added a supplier dataset to enrich the Product dimension.
```

```
# 1. Customer Segment column already exists: 'Segment'

# 2. Modify ETL to fill missing values instead of dropping
df.fillna(0, inplace=True)

# 3. New query: Top region by sales
top_region = df.groupby('Region')['Sales'].sum().idxmax()
print("Top Region by Sales:", top_region)

# 4. Star schema explanation (already visualized earlier)
print("Fact Table: Sales, Profit, Quantity, Discount")
print("Dimensions: Customer (Segment, Region, Country, City, State, Pos

# 5. Add another dataset (suppliers.csv) - example merge
# Suppose suppliers.csv has columns: Category, SupplierName
suppliers = pd.DataFrame({
    'Category': ['Furniture','Office Supplies','Technology'],
    'SupplierName': ['Supplier A','Supplier B','Supplier C']
})
product_dim_extended = product_dim.merge(suppliers, on="Category", how=
print(product_dim_extended.head())
```

```
Top Region by Sales: West
Fact Table: Sales, Profit, Quantity, Discount
Dimensions: Customer (Segment, Region, Country, City, State, Postal Coc
          Category Sub-Category SupplierName
0        Furniture    Bookcases   Supplier A
1        Furniture       Chairs   Supplier A
2  Office Supplies       Labels   Supplier B
3        Furniture       Tables   Supplier A
4  Office Supplies      Storage   Supplier B
```

Executive Summary

This activity explores the design and implementation of a simple data warehouse for a retail company, using a real dataset containing sales, customer, and product information. The objective was to understand and apply key data warehousing techniques, including ETL (Extract, Transform, Load), data modeling, and querying for

techniques, including ETL (Extract, Transform, Load), data modeling, and querying for decision-making.

```
1.Scenario Understanding: We identified three main data sources — sales s

2.ETL Process: We extracted relevant fields, transformed the data by clea

3.Data Modeling: A star schema was designed with a central Sales Fact tab

4.Query & Analysis: We answered key business questions such as total sale

5.Discussion & Reflection: We examined challenges in ETL design, emphasiz

6.Practical Extensions: We added a customer segment column, modified ETL
```

Outcome:

The exercise demonstrates how a well-structured data warehouse enables managers to consolidate information from multiple sources, generate actionable insights, and support strategic decision-making. By applying ETL, schema design, and analytical queries, students gain practical skills in building and exploring a warehouse environment.